PROGRAMACIÓN IV

Iker Diaz, Ian Anaya e Iñigo Fernandez

FASE 0 - ANÁLISIS, DISEÑO Y ARQUITECTURA

1. Especificación de requisitos

El proyecto consiste en un sistema informático de Gestión de la Red de Transporte Público de Bizkaia (Autobuses y Metro). El objetivo principal del sistema es administrar la red de transporte público (líneas, paradas, horarios, incidencias...) y facilitar la consulta de la información por parte del cliente. El sistema consta de 3 partes:

- **Administrador Local:** El administrador local será destinado al personal del consorcio, en el que se podrá administrar de manera local la red de transporte público. Su finalidad será preparar y mantener la información que posteriormente será utilizada por el servidor.

  Desde está aplicación se podrán realizar las siguientes operaciones:
  - Dar de alta nuevas líneas de transporte indicando si pertenecen a bus o metro.
  - Registrar, modificar o eliminar paradas.
  - Carga los datos de la red de transporte público desde archivos en formato CSV a la Base de Datos.
  - Gestionar los horarios de funcionamiento de cada línea.
  - Asociar paradas a una línea concreta manteniendo el orden del recorrido.
  - Registrar incidencias del servicio (retrasos, averías, cortes de línea, etc.)
  - Consultar la información almacenada en la base de datos.
  - Leer los parámetros del sistema desde un fichero de configuración.
  - Registrar las operaciones realizadas en ficheros de log.7

Esta aplicación funcionará de manera local y no tendrá comunicación por red , ya que su objetivo es la administración del sistema.

- **Servidor Remoto:** El servidor remoto actuará como enlace entre la aplicación del cliente y la base de datos.

  El servidor realizará lo siguiente:
  - Abrirá un puerto de red previamente indicado y esperará al cliente a que le envíe peticiones.
  - Atenderá las peticiones de los clientes de uno en uno.
  - Al recibir peticiones del cliente, realizará las consultas necesarias a la base de datos y posteriormente devolverá la o las respuestas necesarias mediante el protocolo elegido.
  - Al realizar cualquier acción o tarea, como puede ser realizar conexiones, consultar datos, etc...; el servidor dejará por escrito todo lo realizado en un archivo propio de logs.

- **Cliente:** El cliente será una aplicación que permitirá al usuario realizar consultas sobre la red de transporte.

  Desde el cliente se podrán realizar las siguientes operaciones:
  - Conectarse al servidor mediante sockets.
  - Consultar las líneas disponibles(bus o metro).
  - Visualizar las paradas de una línea concreta.
  - Consultar horarios de funcionamiento.
  - Ver incidencias activas del servicio.
  - Buscar información de una parada específica.
  - Navegar por la aplicación mediante un menú en modo texto.

El cliente no accede directamente a la base de datos, sino que todas las operaciones se realizan a través del servidor, siguiendo el modelo cliente-servidor.

Sistema de autenticación de Usuarios

El sistema tendrá un sistema de autenticación para controlar el acceso a las aplicaciones y garantizar que únicamente usuarios autorizados puedan utilizar las funcionalidades disponibles.

- Las credenciales de acceso se almacenan en un fichero de texto estructurado (administradores.txt) y serán validadas por la aplicación antes de permitir el acceso.
- Habrá dos roles.:
  - Administrador: puede realizar operaciones de gestión y modificación de datos.
  - Usuario: solo puede realizar consultas a través de cliente.

Esta funcionalidad asegura control de acceso, seguridad básica e integración del uso de ficheros en el sistema.

2. Arquitectura

La arquitectura del sistema se divide claramente en 2 partes. Por una parte el administrador local que se encarga de gestionar el sistema y por otra parte el cliente/servidor que se encargará de las consultas a la base de datos.

El proyecto contará con 3 programas/ejecutables:

1. **Administrador Local**: Este programa será la herramienta de gestión del sistema. Estará completamente desarrollado en C y contará con los siguientes caracteristicas:
   a. La interfaz de usuario será en la consola y tendrá menús para que el administrador pueda gestionarlo de manera cómoda y rápida.
   b. Manejara en memoria la información importada antes de aplicarla o modificarla.
   c. Contará con un módulo de lectura de lectura de archivos para leer las credenciales y los datos de la red.
   d. Tendrá un módulo de lectura, creación, modificación y borrado de datos en la base de datos con los datos proporcionados en los archivos CSV.
   e. También escribirá el rastro de las operaciones realizadas en el programa en un fichero de logs.

2. **Servidor Remoto**: Este programa será el núcleo principal del proyecto en el que se encargará de realizar las consultas solicitadas por el cliente a la base de datos. Estará desarrollado en C y realizará lo siguiente:
    a. Abrirá un puerto de escucha para recibir peticiones de los clientes
    b. Atenderá las peticiones de los clientes de forma secuencial, es decir de una en una.
    c. Usará los mismos archivos de configuración y se conectará a la misma base de datos creada en la Fase 1 para realizar las consultas solicitadas.
    d. Utilizará su propio archivo de logs para auditar todas las operaciones realizadas.
3. **Cliente Remoto**: Será el programa principal que usará el usuario normal para consultar la información sobre líneas, paradas, incidencias, etc... Estará desarrollado en C++ y tendrá las siguientes características:
    a. Menús de consola (parecidas a las del Administrador Local), adaptados para la consulta del viajero.
    b. Establecimiento de comunicación con el servidor para solicitar la información necesaria mediante sockets.
    c. Implementará estructuras de datos complejas en memoria para almacenar los datos ya solicitados al servidor. De esta forma, se minimizan las peticiones por la red; si un usuario consulta las paradas de la "Línea 1" dos veces, la segunda vez se leerán de la memoria local del cliente.
    d. Tendrá su propio archivo de logs para recolectar todas las acciones realizadas.

3. Interfaz de Usuario e Informes y Secuencia de Interacción

El sistema utiliza una interfaz en modo terminal, tanto en el Administrador Local como en el Cliente Remoto, con menús jerárquicos y formularios de entrada para facilitar la interacción del usuario.

Menú del Administrador Local

Al iniciar la aplicación se solicita:

- Usuario
- Contraseña

Validación de credenciales:

- Solo se permite el acceso si el usuario está guardado en el fichero de administradores.
- Máximo 3 intentos fallidos → cierre de la aplicación.

Menú Principal del Administrador:

1. **Importar paradas y líneas (CSV)**
   Permite cargar información desde archivos CSV a la base de datos.
2. **Gestionar Red → Submenú Gestión de Red**
   Permite añadir, modificar o eliminar líneas paradas.
   2.1. Añadir línea
   2.2. Modificar línea
   2.3. Borrar línea
   2.4. Añadir parada
   2.5. Modificar parada
   2.6. Borrar parada

2.7. Volver al Menú Principal

3. **Panel de Incidencias → Submenú Incidencias**
   Permite registrar alertas o incidencias en líneas o paradas (retrasos, averías, cortes).
   3.1. Añadir incidencias a línea
   3.2. Añadir incidencias a parada
   3.3. Consultar incidencias
   3.4. Volver al Menú Principal
4. **Salir**
   Finaliza la sesión de manera controlada.

Menú del Cliente Remoto

El Cliente permite a los usuarios consultar información de la red de transporte de forma remota.

- Al ejecutar el programa, no se solicita login para un usuario normal; se accede directamente al menú.
- El programa está pensado para usuarios que solo realizan consultas de información

Menú Principal del Cliente:

1. **Buscar líneas por Municipio**
   Ejemplo: Introducir "Basauri" → muestra todas las líneas de bus y metro que operan en ese municipio.
   1.1. Filtrar solo líneas de bus
   1.2. Filtrar solo líneas de metro
   1.3. Quitar filtro
   1.4. Volver al Menú Principal
2. **Consultar itinerario de una Línea**
   Ejemplo: Introducir una línea " 1 " → muestra el menú de consultas para esa línea
   2.1. Ver todas las paradas en orden
   2.2. Buscar información de una parada concreta
   2.3. Introducir la nueva línea
   2.4. Volver al Menú Principal
3. **Ver estado de la red**
   Muestra avisos e incidencias activar en la red.
   3.1. Ver información de incidencias activas en toda la red
   3.2. Ver información de incidencias por línea
   3.3. Ver incidencias entre parada origen y destino
   3.4. Volver al Menú Principal
4. **Salir**
   Finaliza la sesión de forma segura.

Informes y Visualización de Datos

Todos los informes se mostrarán en terminal de manera tabulada y formateada, y opcionalmente se guardarán en archivos de texto.

Ejemplos de informes:

1. Listado de líneas filtradas por Municipio (Ej. "Basauri"):

```
None
CÓDIGO || TIPO  || NOMBRE (RECORRIDO DE LA LÍNEA)
==========================================================
L2     || METRO || Basauri - Kabiezes
A3613  || BUS   || Bilbao - Ugao-Miraballes
A3622  || BUS   || Bilbao - Zaratamo
```

2. Consulta de Próximos Horarios en una Parada:

```
None
PARADA: San Mamés (Accesible: SÍ)
LÍNEA || TIPO DÍA  || PRÓXIMAS SALIDAS
==========================================================
L1    || Laborable || 08:05, 08:12, 08:18, 08:24
A3136 || Laborable || 08:15, 08:45, 09:15
```

3. Panel de Incidencias Activas en la Red:

```
None
LÍNEA || FECHA      || DESCRIPCIÓN DEL AVISO
==========================================================
L1    || 01-03-2026 || Retrasos de 5 min por avería técnica
A3136 || 01-03-2026 || Parada Moyua anulada por eventos deportivos
L2    || 02-03-2026 || Estación Abando cerrada temporalmente
```

4. Reporte interno del Servidor (Fichero `server.log`):

```
None
[18-02-2026 10:15:00] [INFO] Nueva conexión aceptada (Cliente_ID:
104)
[18-02-2026 10:15:05] [RECV] Comando recibido:
GET_LINEAS_MUNICIPIO|Basauri|BUS
[18-02-2026 10:15:05] [SEND] Respuesta enviada: OK (2 resultados)
[18-02-2026 10:16:10] [INFO] Cliente_ID: 104 desconectado. Socket
cerrado.
```

4. Estructura de ficheros

El sistema utilizará distintos tipos de ficheros para configuración, autentificación, carga de datos y registro de actividad.

Ficheros de Configuración

1. **config.txt**

   Contendrá los parámetros generales del sistema, leídos tanto por el administrador como por el servidor.

   Formato: Clave = valor

   Información que guarda:

   - Datos de conexión a la base de datos.
   - Puerto del servidor.
   - Parámetros generales del sistema.

Ficheros de Usuarios

1. **administradores.txt**

   Almacena las credenciales para autenticación.

   Formato: usuario;password

   Información que guarda:

   - Nombre de usuario.
   - Contraseña

   Este fichero es leído por:

   - Administrador Local.

Ficheros CSV de Importación

Utilizados únicamente por el administrador Local.

1. **municipios.csv**

   Almacena los municipios del sistema.

   Formato: ID_Municipio;Nombre;Provincia

   Información que contiene:

   - ID_Municipio: identificador único del municipio (PK)
   - Nombre: nombre del municipio
   - Provincia: nombre de la provincia

2. **paradas.csv**

   Contiene todas las paradas de la red.

   Formato: ID_Parada;Nombre;Tipo;Accesible;ID_Municipio

   Información que contiene:

   - ID_Parada: identificador único (PK).
   - Nombre: nombre de la parada.
   - Tipo: tipo de parada (BUS / METRO).
   - Accesible: si es accesible (SI / NO).
   - ID_Municipio: municipio al que pertenece (FK).

3. **lineas.csv**

Guarda la información básica de cada línea de transporte.

Formato: ID_Linea;Codigo;Nombre;Tipo

Información que contiene:

- ID_Linea: identificador único (PK)
- Código: código corto.
- Nombre: nombre descriptivo de la línea.
- Tipo: BUS / METRO.

4. linea_parada.csv

Define el recorrido de cada línea relacionándola con sus paradas y especificando el orden del trayecto.

Formato: ID_Linea;ID_Parada;Orden_Trayecto

Información que contiene:

- ID_Linea: (FK)
- ID_Parada: (FK)
- Orden_Trayecto: posición de esa parada dentro del recorrido de la línea.

5. horarios.csv

Almacena los horarios de paso de cada línea por cada parada.

Formato: ID_Horario;Hora_Paso;Tipo_Dia;ID_Linea;ID_Parada

Información que contiene:

- ID_Horario: identificador único del horario (PK).
- Hora_Paso: hora de paso (HH:MM).
- Tipo_Dia: tipo de día (Laborable / Festivo / FinSemana...).
- ID_Linea: línea a la que aplica (FK).
- ID_Parada: parada donde aplica (FK).

6. incidencias.csv

Recoge las incidencias del servicio asociadas a una línea.

Formato: ID_Incidencia;Tipo;Descripcion;Fecha;Hora;Activa;ID_Linea

Información que contiene:

- ID_Incidencia: identificador único (PK).
- Tipo: tipo de incidencia (Retraso, Avería, Corte..).
- Descripción: texto descriptivo.
- Fecha: YYYY-MM-DD.
- Hora: HH:MM.
- Activa: SI / NO.
- ID_Linea: línea afectada (FK).

Ficheros de Logs

Cada programa tendrá su propio fichero de log.

1. admin.log

Registra:

- Altas, bajas y modificaciones.
- Importaciones CSV.

- Errores.
- Intentos de login.

2. **server.log**

   Registra:
   - Conexiones de clientes.
   - Peticiones recibidas.
   - Consultas realizadas.
   - Errores de red o base de datos.

3. **cliente.log**

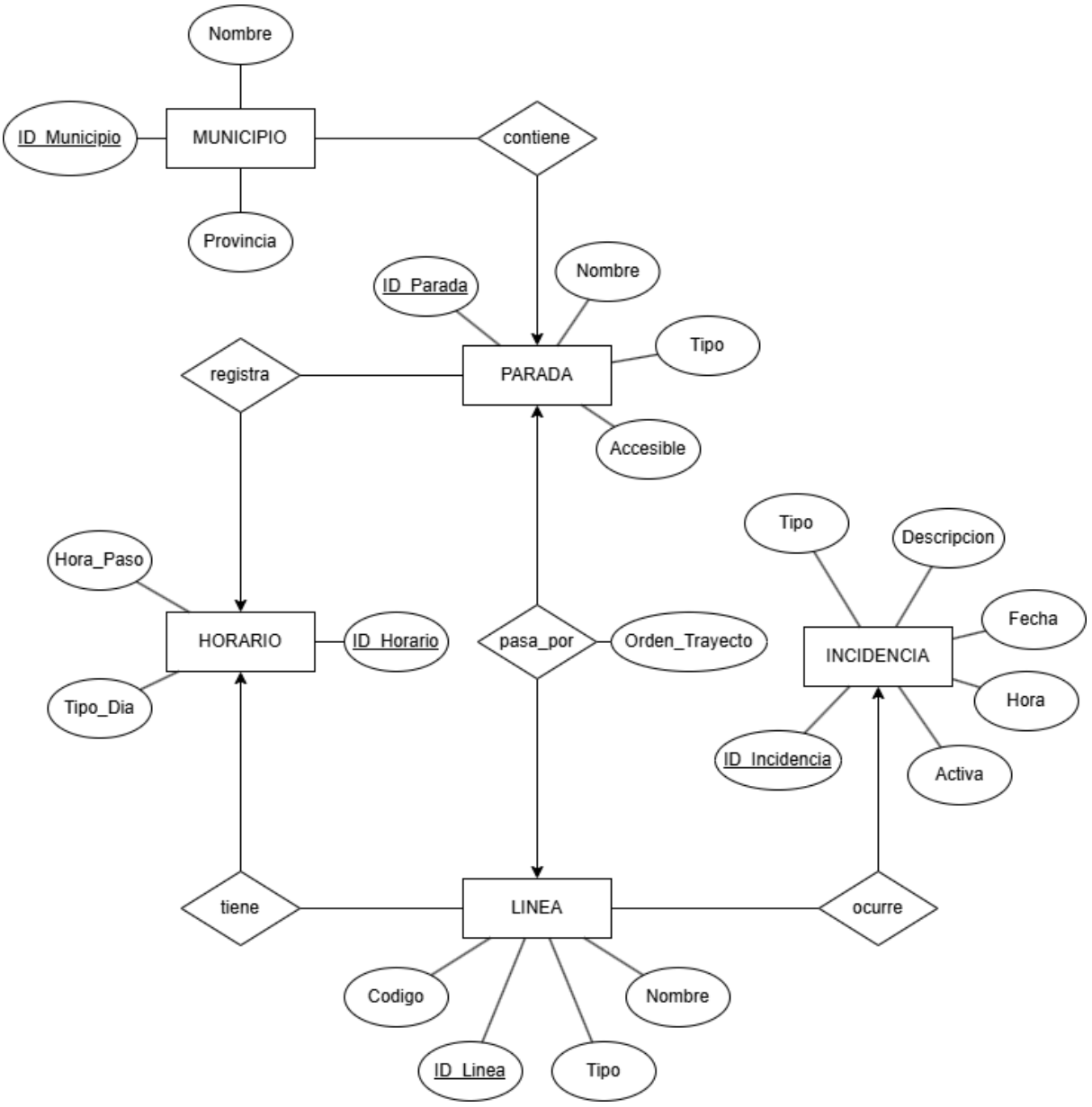
   Registra:
   - Conexiones al servidor.
   - Consultas realizadas.
   - Errores de comunicación.

Base de datos

Es la base de datos donde se almacenan todas las tablas del sistema de transporte público.

1. **transporte_bizkaia.db**

5. Entidad-Relación de la base de datos



6. Protocolo de comunicaciones

La comunicación mediante Sockets utilizará mensajes de texto plano delimitados por el carácter "‖" para separar parámetros y ";" para separar listas de datos. Al ser un cliente público, no requiere comando de autenticación; el usuario se conecta y comienza a consultar directamente.

- **Formato de Petición:** COMANDO ‖ Parametro1 ‖ Parametro2
- **Formato de Respuesta:** ESTADO ‖ Dato1;Dato2... (Donde ESTADO puede ser OK o ERR).

Diccionario de Comandos:

1. Búsqueda de líneas por Municipio

- Comando: GET_LINEAS_MUNICIPIO || <Nombre_Municipio> || <Filtro>
    - *Filtro* puede ser: BUS, METRO o ALL.
- Ejemplo de Petición (Cliente): GET_LINEAS_MUNICIPIO || Basauri || BUS
- Ejemplo de Respuesta (Servidor): OK || A3613, Bilbao-Ugao; A3622, Bilbao-Zaratamo

2. Consultar itinerario de una Línea

- 2.1. Ver todas las paradas en orden:
    - Comando: GET_ITINERARIO || <ID_Linea>
    - Petición: GET_ITINERARIO || L1
    - Respuesta: OK || 1, Etxebarri; 2, Bolueta; 3, Basarrate... *(Lista ordenada)*
- 2.2. Buscar información de una parada concreta (y sus próximos horarios):
    - Comando: GET_INFO_PARADA || <ID_Linea> || <Nombre_Parada>
    - Petición: GET_INFO_PARADA || L1 || Basarrate
    - Respuesta: OK || Accesible: SI; Proximos_Horarios: 08:05, 08:12, 08:18...

3. Ver estado de la red e incidencias

- 3.1. Ver incidencias activas en toda la red:
    - Comando: GET_INCIDENCIAS_GLOBAL
    - Petición: GET_INCIDENCIAS_GLOBAL
    - Respuesta: OK || L1: Retrasos 5 min; A3136: Desvío por obras
- 3.2. Ver incidencias por línea:
    - Comando: GET_INCIDENCIAS_LINEA || <ID_Linea>
    - Petición: GET_INCIDENCIAS_LINEA || L1
    - Respuesta: OK || 2026-03-01: Retrasos 5 min por avería técnica
- 3.3. Ver incidencias entre parada origen y destino:
    - Comando: GET_INCIDENCIAS_TRAYECTO || <ID_Linea> || <Parada_Origen> || <Parada_Destino>
    - Petición: GET_INCIDENCIAS_TRAYECTO || L1 || Etxebarri || Moyua
    - Respuesta: OK || Estacion Abando cerrada, los trenes no efectuan parada.

4. Cierre de sesión y Errores

- Salir / Desconexión:
    - Petición: LOGOUT
    - Respuesta: BYE *(El servidor procede a cerrar el socket de ese cliente).*
- Respuestas de Error estandarizadas (Servidor -> Cliente):
    - ERR || NO_DATA (La consulta se ejecutó bien en la base de datos, pero no hay resultados. Ej: municipio sin líneas de metro).
    - ERR || BAD_REQUEST (Parámetros incorrectos, faltan delimitadores || o la línea introducida no existe).